

Off by 1 - Canceling async Redis command leaves connection open, in unsafe state for future commands #2624

Closed

#2641

drago-balto opened on Mar 17, 2023 · edited by drago-balto

Edits ...

Version: 4.5.1**Platform:** Python 3.8 / Ubuntu (but really, any platform will likely suffer the same issue)**Description:** Canceling async Redis command leaves connection open, in unsafe state for future commands

The issue here is really the same as [#2579](#), except that it generalizes it to all commands (as opposed to just blocking commands).

If async Redis request is canceled at the right time, after the command was sent but before the response was received and parsed, the connection is left in an unsafe state for future commands. The following redis operation on the same connection will send the command, and then promptly continue to read the response from the previous, canceled command. From that point on, the connection will remain in this weird, off-by-1 state.

Here's a script reproducing the problem:

```
import asyncio
from redis.asyncio import Redis
import sys

async def main():
    myhost, mypassword = sys.argv[1:]
    async with Redis(host=myhost, password=mypassword, ssl=True, single_connection_client=True) as r:

        await r.set('foo', 'foo')
        await r.set('bar', 'bar')

        t = asyncio.create_task(r.get('foo'))
        await asyncio.sleep(0.001)
        t.cancel()
        try:
            await t
            print('try again, we did not cancel the task in time')
        except asyncio.CancelledError:
            print('managed to cancel the task, connection is left open with unread response')

    print('bar:', await r.get('bar'))
    print('ping:', await r.ping())
```

```
print('foo:', await r.get('foo'))
```

```
if __name__ == '__main__':  
    asyncio.run(main())
```

Running this against a server in the cloud (such that requests typically take > 5ms to complete) results in this:

```
$ python redis_cancel.py hostname password  
managed to cancel the task, connection is left open with unread response  
bar: b'foo'  
ping: False  
foo: b'PONG'
```



I believe the solution is simple. [This method](#) needs to be modified to disconnect connection when current request is canceled. For example, do this:

```
async def _send_command_parse_response(self, conn, command_name, *args, **options)  
    """  
    Send a command and parse the response  
    """  
    try:  
        await conn.send_command(*args)  
        return await self.parse_response(conn, command_name, **options)  
    except asyncio.CancelledError:  
        await conn.disconnect()  
        raise
```



11

4



 **chayim** mentioned this [on Mar 23, 2023](#)

[AsyncIO Race Condition Fix #2641](#)



 **chayim** closed this as [completed](#) in [#2641](#) [on Mar 23, 2023](#)



 **chayim** mentioned this in 2 pull requests [on Mar 23, 2023](#)

[AsyncIO Race Condition Fix #2640](#)

[AsyncIO Race Condition Fix #2639](#)

 **volnet** on Mar 25, 2023



Is it the problem about OpenAI <https://openai.com/blog/march-20-chatgpt-outage> ?

3

7

 **drago-balto** on Mar 25, 2023

Author



Yep, that's the one, and the [#2641](#) has not fixed it fully, as I already commented here: [#2641 \(comment\)](#)

I am asking for this ticket to be re-oped, since I can still reproduce the problem in the latest 4.5.3. version

5

rajivpoddar on Mar 25, 2023

Did GPT4 help with debugging?

14

33

71

2

12



scovetta mentioned this [on Mar 25, 2023](#)

✓ [CVE Request for Async Connection Bugs #2664](#)

pydanny on Mar 25, 2023 · edited by pydanny

Edits ...

I wonder how much OpenAI/ChatGPT is funding maintenance of this library. 🤖

35

20

7



drago-balto mentioned this [on Mar 26, 2023](#)

✓ [Canceling async Redis command leaves connection open, in unsafe state for future commands #2665](#)



chayim reopened this [on Mar 26, 2023](#)

chayim on Mar 26, 2023

Contributor ...

Folks, I reopened this accidentally - but wanted to ensure there was communication that is clear. Please note - we're tracking in [#2665](#). Let's align in a single issue - it makes it easier to co-ordinate, track, and respond.



chayim closed this as [completed](#) [on Mar 26, 2023](#)



kurtseifried mentioned this [on Mar 30, 2023](#)

✓ [affected\[\].ranges\[\].type missing timestamp type ossf/osv-schema#129](#)



kristjanvalur mentioned this in 2 pull requests [on Apr 5, 2023](#)

🔗 [Optionally disable disconnects in read_response #2695](#)

🔗 [Revert #2104, provide way to disable disconnects in read_response\(\) #2506](#)



Chronial mentioned this [on May 8, 2023](#)

✓ [BaseException at I/O corrupts Connection #2499](#)

rgaufman on Jun 11, 2023

This is quite a critical bug that is impacting us, can you please release a new version?

tomer555 on Jun 24, 2023

+1



Petitoto added a commit that references this issue on Aug 19, 2023

Bump redis from 4.6.0 to 5.0.0 ([#227](#))

Verified 5d14600



oleobal added 2 commits that reference this issue on Sep 12, 2023

chore(deps): bump redis from 4.5.4 to 5.0.0 in /backend ([#724](#))

Verified 64e54bd

chore(deps): bump redis from 4.6.0 to 5.0.0 in /metrics-exporter ([#72](#))

Verified 75fd948



github-actions mentioned this in 2 issues on May 7, 2024

[LOW: redis - redis: Async command information disclosure FabioBartoli/Linuxtips-telegram-communication#61](#)

[LOW: redis - redis: Async command information disclosure FabioBartoli/Linuxtips-telegram-communication#64](#)

[Sign up for free](#) to join this conversation on GitHub. Already have an account? [Sign in to comment](#)

Assignees

No one assigned

Labels

No labels

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

 AsyncIO Race Condition Fix

redis/redis-py

Participants

